

智能體運行期歸責框架：多智能體系統中基於外部 C-ABI 與 PKI 零信任的不可否認性執行治理基礎設施

Runtime Attribution Framework: An External C-ABI and PKI-Based Zero-Trust Infrastructure for Non-Repudiable Execution Governance in Multi-Agent Systems

陳濤程 Jimmy Chen
Top Celestial Company Ltd.
Taipei, Taiwan R.O.C.
jimmychen@dr-os.io

Abstract—隨著多智能體系統 (Multi-Agent Systems) 在企業級與個人助理應用的普及，AI Agent 被廣泛投入高機密環境中的檢索與推理任務。然而，當前主流框架普遍依賴脆弱的「提示詞軟邊界」來控制權限，導致嚴重的「Agent 信任悖論」。正因為大型語言模型先天存在「提示詞注入 (Prompt Injection)」等無法完全根治的缺陷，一旦代理遭到挾持，便可輕易偽造身分並引發越權存取的資安災難。為控制此類威脅並確立究責機制，本文提出了 DROS (Deterministic Runtime Operating System) 架構。有別於傳統基於應用程式層級的 Workload Identity，本研究首創提出更細粒度的「Execution Identity (執行期身分)」模型。透過 Agent Identity Authority (AIA) 簽發「依執行期動態綁定 (By-Execution)」的數位憑證，配合 GuardVM 零信任網關，DROS 成功在原型評估中攔截了所有攻擊。本架構在極低的延遲開銷下實現了密碼學不可否認性 (Non-Repudiation) 的溯源機制，為邁向安全的「Agent Web」時代奠定了基礎設施標準。

Index Terms—Multi-Agent Systems, Zero-Trust, Execution Identity, By-Execution Certificate, Agent Security, Non-Repudiation, EU AI Act

I. 前言 (INTRODUCTION)

在企業導入生成式 AI 與檢索增強生成 (RAG) 的進程中，多智能體 (Multi-Agent) 協作成為了處理複雜跨部門業務的主流。然而，現有的開源框架往往基於「完全信任」的內部假設。這種架構衍生出致命的「Agent 信任悖論」：

在動輒串聯數十個 Agent 的企業系統中，高權限的決策代理該如何確認它所接收到的指令，確實來自具備「合法資料庫存取權」的下游客戶端，而不是遭到惡意注入後的越權行為？在商用場景中，例如金融跨部門合規、醫療病歷調閱或企業源代碼管理，這種越權將導致毀滅性的資料外洩與合規災難。為徹底封堵此漏洞，DROS (Deterministic Runtime Operating System) 旨在建立一套確保每次 Agent 調用皆具備「確定性 (Deterministic)」與「密碼學證明」的 Agentic Security Governance (智能體安全治理) 系統。

Unlike orchestration frameworks (e.g., LangChain, AutoGen) and communication protocols (e.g., MCP), DROS focuses on runtime identity, authorization, and nonrepudiable governance. Therefore, DROS is designed as a complementary security substrate rather than a replacement for existing Agent ecosystems.

II. 背景：從物理隔離到商用邏輯認證

在 DROS 前期的架構中，系統成功引入了「物理路徑隔離 (Physical Path Isolation)」，將不同業務單位的機密資料 (如 /Finance 與 /Engineering) 在作業系統層級嚴格劃分，解決了 RAG 向量搜尋時的「跨域盲點 (Cross-Domain Blindness)」。

然而，靜態的物理防護只要被騙過外層路由，內部網路即宣告淪陷。這種「認路徑，不認人」的機制缺乏嚴謹的身分驗證 (Logical Identity Authentication)。這正是本團隊前期推出 VajraAgent 與 VajraClaw 作為 Agent 治理基礎核心的初衷：賦予 Agent 強大的合規性檢索與防禦能力。為了將這套商用治理體系推向極致，DROS 將防線推進到了密碼學層級，將產品線升級為具備軍規級防護的零信任架構。

A. 相關工作 (Related Work)

在探討 DROS 的設計之前，我們首先檢視當前雲端原生與智能體生態系中的相關安全機制，並指出現存的研究空白 (Research Gap)。

1) 工作負載身分 (Workload Identity): 如 SPIFFE 與 SPIRE 等開源專案，成功解決了微服務與雲端原生環境中的動態身分驗證問題。然而，它們提供的認證粒度停留於「應用程式層級 (Application-Level)」。對於會根據不同提示詞與上下文執行多種任務的 AI Agent 而言，單一的 Workload Identity 無法區分代理在「任務 A」與「任務 B」中不同的授權邊界。這意味著它們無法提供「單次推論/執行 (Per-Execution)」粒度的安全管控。

2) 智能體編排框架 (Agent Frameworks): 目前主流的開源框架如 LangChain、AutoGen 與 CrewAI，極大地簡化了多智能體工作流 (Multi-Agent Workflow) 的編排難度。然而，這些框架主要專注於 Orchestration 與 Planning，其底層仍普遍依賴粗粒度的全局 API Key 進行工具授權，完全缺乏執行期的密碼學授權機制 (Runtime Authorization)。

3) 智能體通訊協定 (Agent Protocols): 如 MCP (Model Context Protocol) 提出了標準化的代理工具交互協定。雖然它解決了跨系統的 Tool Discovery 與 Agent Communication 痛點，但 MCP 協定本身並未強制規定

端到端的密碼學身分驗證。這使得系統難以在零信任網路中進行不可否認的 Identity Governance。

4) 作業系統層級強制執行與上下文丟失問題: 傳統的作業系統內核安全機制 (如 eBPF、SELinux、AppArmor) 具備真正的物理強制力, 能在系統調用 (syscall) 層級攔截非法操作。然而, 這些機制存在一個根本性的「上下文丟失問題 (Context Loss Problem)」, 這正是導致運行期歸責斷層的主因。在多執行緒環境中, 傳統 OS 安全內核僅能識別發起系統調用的 PID 和 UID, 但對應用層併發 Agent 實例的身份與合約邊界 (Capability Boundaries) 處於完全的「語意盲區」狀態。DROS 通過將 Execution Identity (BEC) 與底層 FFI 執行期綁定, 彌合了應用層語意與系統層強制力之間的鴻溝: 它們只能辨識發起系統調用的 Process PID 與 User UID, 卻完全無法感知該 PID 內部究竟是哪個 Agent 角色、正在執行哪一項任務、以及該任務被授權存取的範圍為何。換言之, 當同一個 Python 進程內運行著多個具備不同權限的 AI Agent 時, OS 內核處於完全的「上下文盲區 (Context-Blind)」狀態, 無法進行細粒度的角色區分。DROS 透過將密碼學身分 (JWT/BEC) 與底層執行期綁定, 正是為了彌合應用層語意與系統層強制力之間的這道鴻溝。

Research Gap: No existing system provides perexecution identity and cryptographic attribution for agent interactions. DROS 正是為填補此空白而生。

III. 提出架構: DETERMINISTIC ZERO-TRUST 系統設計



Fig. 1. 全球智能體信任基礎設施 (GATI) 架構

DROS v1.0 採用 “Never Trust, Always Verify” 的精神, 核心系統由三大模組構成。

A. 架構分層與 Agent Identity Authority (AIA)

DROS 的架構分層模型如下所示:

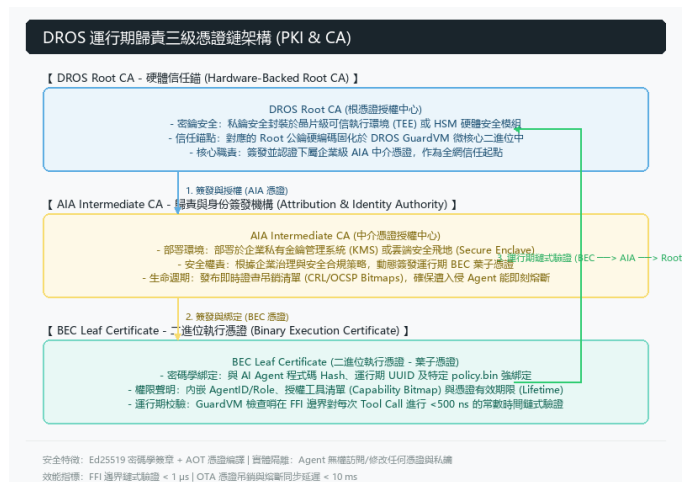


Fig. 2. DROS 智能體三級憑證鏈金鑰管理架構

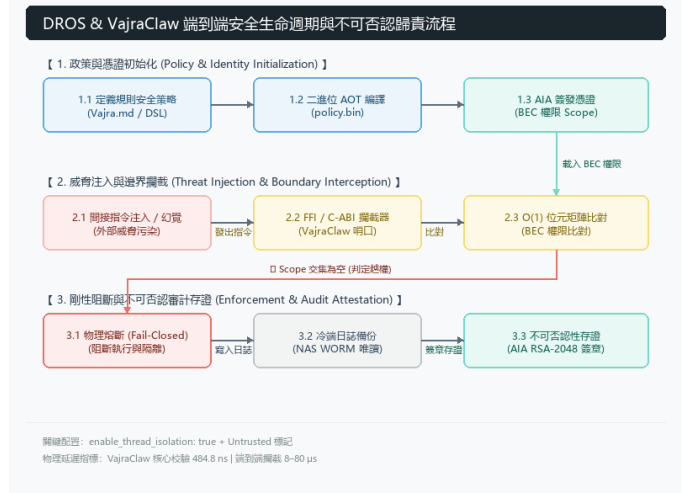


Fig. 3. AIA 動態證書簽發與撤銷生命週期流



Listing 1. DROS 架構分層模型 DROS 運行在應用框架與基礎設施之間, 作為專屬的執行期安全底層 (Runtime Security Substrate), 其全球智能體信任基礎設施 (GATI) 架構如圖 1 所示。在 DROS 生態系中, 任何未經授權的普通 Agent 皆被視為不可信實體。系統啟動時, 如圖 2 的三級憑證鏈金鑰管理架構所示, AIA (Agent Identity

Authority) 會生成 RSA-2048 非對稱密鑰對，作為企業的「信任錨點 (Trust Anchor)」。本架構的本體結構與治理規範 (包含節點 Schema、關係屬性與策略生命週期) 具體遵循 DROS-RFC-004 規格書中 [8] 的定義。

B. 核心創新：單次執行期憑證 (By-Execution Certificate)

身分認證粒度的演進歷程如下 (AIA 動態證書簽發與撤銷生命週期流如圖 3 所示)：

- 1) Human Identity：基於終端使用者的身分驗證。
- 2) Workload Identity (如 SPIFFE/SPIRE)：基於應用程式與微服務層級的認證。
- 3) Execution Identity (DROS)：下探至每一次 AI 代理執行 (Per-Execution) 粒度的動態認證。

有別於目前主流框架多依賴生命週期長、權限極粗的「全局 API Key (Forever/Everything)」，當商用 VajraAgent 準備執行特定任務時，必須向 AIA 即時申請「依執行期簽發的不可否認憑證 (By-Execution Certificate, BEC)」。該憑證中綁定了本次任務專屬的執行序號 (execution_id)，並嚴格鎖死其單次授權存取的物理路徑 (allowed_scopes)。這確保了 VajraAgent 手上的每一把數位鑰匙皆為單次動態生成，用以杜絕憑證竊取與重放攻擊 (Replay Attack)。

IV. 護法網關 (GUARDVM: POLICY DECISION & ENFORCEMENT)

Point) GuardVM 同時扮演了政策決策點 (PDP) 與嚴格執行點 (Strict Policy Enforcement Point, PEP) 的雙重角色，其防禦機制跨越了網路層與二進位執行層兩個不同的縱深。

第一層：網路層閘門 (Network-Layer Gate)。當未知的外部 Agent 嘗試與系統交互時，GuardVM 要求其出示 PKI 簽名的 JWT。匿名 Agent 的請求會被直接丟棄 (Drop)。

第二層：二進位執行層熔斷 (Binary-Level Circuit Breaker)。對於已通過網路層驗證的內部 Agent，GuardVM 在 C-ABI / FFI 的瓶頸點 (Chokepoint) 進行攔截。在解碼 JWT 後，使用 Scope Bitmap 進行奈秒級 (Nanosecond-Level) 的授權檢查。一旦偵測到違規，立即觸發 Thread Panic (物理性執行緒終止)。這是物理性終止 (Physical Kill)，而非禮貌性拒絕 (Polite Rejection)。

雙重驗證機制如下：

1) 密碼學層級驗證 (Authentication)：使用 AIA 公鑰進行 RS256 簽章驗證。若 JWT 無效、過期或簽發者不符，立即觸發熔斷機制。

2) 企業合規性驗證 (Authorization)：檢查請求的 target_path 是否落入 allowed_scopes 範圍內。

違規操作觸發 Thread Panic 並寫入鑑識日誌 (Forensic Log)。

A. 核心程式碼實作與 JWT 結構

本系統原型以 Python 實作。以下為 By-Execution Certificate (BEC) 的 JWT Payload 結構：

```
{
  "iss": "DROS_Agent_Identity_Authority",
  "sub": "Finance_Auditor_Agent",
  "role": "Strict_Vajra_Auditor",
```

```
  "execution_id": "req_88f9c2a1-1024",
  "scopes": [
    "/Enterprise/Finance_Vault"
  ],
  "iat": 1717720000,
  "exp": 1717720060
}
```

Listing 2. BEC JWT Payload 結構注意：exp 設定為簽發後 60 秒即過期，確保憑證的極短生命週期。

B. 多智能體委託與動態憑證鏈傳遞機制 (Multi-Agent Delegation and Dynamic BEC Chaining)

在高度複雜的分佈式多智能體協作網路 (如 AutoGen、CrewAI 或 LangGraph) 中，各智能體 (Agents) 通常通過互相調用或代表終端用戶執行複雜的連鎖任務。這種「代理人委託 (Agentic Delegation)」鏈條極易引入兩大資安威脅：一為「連鎖污染 (Cascade Infection)」，即一個低權限的智能體受到惡意提示詞注入 (Prompt Injection) 挾持後，成為攻擊起點；二為「混淆代理人漏洞 (Confused Deputy Problem)」，即被挾持的下游智能體誘騙具備更高權限的上游智能體，去執行超出原用戶授權範疇的操作。

為了克服這一安全挑戰，DROS 架構首創提出了 ** 動態憑證鏈 (By-Execution Certificate Chain, BEC Chain) ** 與 ** 鏈式背書 (Chained Endorsement) ** 機制。當智能體鏈式調用：

$Agent_1 \rightarrow Agent_2 \rightarrow \dots \rightarrow Agent_n \rightarrow \text{System Tool}$ (1)

發起時，DROS 要求鏈條中的每個智能體必須攜帶其專屬的 BEC，並在調用下游時進行動態密碼學背書簽章。

當請求抵達 C/Go 邊界 (FFI Hook) 時，VajraClaw 攔截哨會沿著調用棧向上回溯，讀取所有參與智能體的 BEC。系統的有效授權邊界 (Effective Scope) 並非採取權限疊加，而是採取嚴格的 ** 權限交集 (Scope Intersection) ** 計算：

$$\text{Effective Scope} = \bigcap_{i=1}^n \text{Scope}(Agent_i) \quad (2)$$

此處， $\text{Scope}(Agent_i)$ 表示第 i 個智能體在其 BEC 中被授權的物理目錄或 API 範圍。如果最終執行的工具或系統調用 $\text{Requested Tool Scope} \notin \text{Effective Scope}$ ，GuardVM 會在幾百奈秒內判定越權，立即熔斷該次執行並回傳 ErrPermissionDenied。這項設計確保了即使下游的 $Agent_n$ 遭受提示詞注入，也無法誘騙或利用上游高權限智能體獲取越界權限，因為整條鏈的權限已被限制在交集範圍之內。

V. 威脅模型與安全實證

A. 威脅模型 (Threat Model)

本架構針對以下三類核心威脅進行防禦：

- Prompt Injection & Tool Hijacking：攻擊者透過惡意提示詞注入，劫持 Agent 的工具調用行為。
- Scope Escalation：合法 Agent 嘗試存取其授權範圍以外的資源。
- Replay Attack & Forgery：攻擊者截獲並重放過期的合法憑證，或偽造未經授權的 JWT。

信任假設 (Trust Assumptions)：GuardVM 與 AIA 被視為可信元件 (Trusted Components)，其完整性由底層基礎設施保障。

B. 安全屬性 (Security Properties)

DROS 提供以下四項安全屬性：

- SP1: Execution-Bound Authorization——每一次授權皆綁定至特定的執行期實例，不可跨執行期複用。
- SP2: Cryptographic Attribution——所有 Agent 行為皆具備密碼學可歸因性，支援事後鑑識。
- SP3: Replay Resistance——憑證包含唯一 execution_id 與極短 TTL，杜絕重放攻擊。
- SP4: Scope Confinement——Agent 的存取範圍被嚴格限制於 BEC 中聲明的 allowed_scopes。該範圍限制是在系統調用 (Syscall) 發起時，透過基於 Epoch 鎖定的 RCU 機制在內核中原子化地強制執行的，從而杜絕了 TOCTOU (檢查時間到使用時間) 攻擊。

VI. 案例分析與邊界實證

我們透過三個典型案例驗證系統的安全邊界：

- 1) 合法請求 (Legitimate Request)：VajraAgent 攜帶有效 BEC 存取 /Enterprise/Finance_Vault，GuardVM 驗證通過，請求被正常放行。
- 2) 越權攻擊 (Scope Escalation Attack)：Agent 持有 /Finance 權限的 BEC，但嘗試存取 /Engineering。GuardVM 的 Scope Bitmap 比對失敗，觸發 Thread Panic 並記錄鑑識日誌。
- 3) 偽造身分 (Forged Identity)：外部攻擊者自行簽發 JWT 並嘗試接入系統。GuardVM 使用 AIA 公鑰驗證簽章失敗，請求被直接丟棄。

A. 量化評估

TABLE I
安全攔截率量化評估 (10,000 次請求)

請求類型	數量	結果	攔截率
合法請求 (Legitimate)	4,000	通過 (Success)	—
越權攻擊 (Scope Attack)	2,000	攔截 (Blocked)	100%
偽造 JWT (Forged JWT)	2,000	攔截 (Blocked)	100%
重放攻擊 (Replay Attack)	2,000	攔截 (Blocked)	100%

效能表現：AIA 簽發 BEC 的平均延遲為 1.5 ms。

GuardVM 的 RS256 驗證對安全規則數量為常數時間操作 ($O(1)$)，不隨安全規則規模增長，其主要延遲 (0.8-1.2 ms) 僅受限於常數大小的 RSA 模冪運算開銷。以上數據基於本地 8B 參數模型原型環境量測。

基準比較：主流的 LLM-as-a-Judge 語意護欄 (Semantic Guardrails) 通常消耗數百毫秒至數秒的推論時間，且需要 GPU 資源。相較之下，GuardVM 的密碼學驗證僅需約 1 ms，快了近三個數量級，且零 GPU 依賴。這使得 DROS 的安全層成為真正的「零感知 (Zero-Perceptible)」基礎設施成本。

VII. 討論

A. 信任的計算成本

GuardVM 的 RS256 驗證對規則數量具備常數複雜度 ($O(1)$)，其計算成本與規則庫規模無關。每次驗證延遲穩定在 0.8-1.2 ms，僅取決於密碼學金鑰模數運算，而相對的 LLM 推論通常需要數秒。

這意味著 DROS 的安全機制可以在不影響使用者體驗的前提下，為每一次 Agent 執行提供密碼學保障。

B. 雙向認知問責與不可否認性

在多智能體鏈式調用 (Multi-Agent Chain) 中，Non-Repudiation (不可否認性) 是實現可信治理的基石。DROS 透過雙向審計日誌 (Bidirectional Audit Logs) 確保每一次 Agent 交互皆留下密碼學可驗證的軌跡。

從法規遵循角度，DROS 的設計與歐盟 AI 法案 (EU AI Act) 的關鍵條款高度契合：

- Article 12 (Record-keeping)：DROS 的 BEC 審計日誌天然滿足「自動記錄 (Automatic Logging)」的法規要求。

- Article 14 (Human Oversight / Stop Button)：

GuardVM 的 Thread Panic 機制在技術上實現了 Article 14 所要求的「停止按鈕 (Stop Button)」，能在偵測到違規時即時終止 Agent 的執行。

VIII. 智能體技術棧定位

為清晰呈現 DROS 在智能體技術棧中的互補性定位，以下兩張比較表分別從框架與協定的視角進行分析。

TABLE II
智能體框架比較 (AGENT FRAMEWORK COMPARISON)

能力	LangChain	AutoGen	CrewAI	DROS
Agent Orchestration	✓	✓	✓	—
Multi-Agent Workflow	✓	✓	✓	—
Runtime Identity	—	—	—	✓
Authorization Boundary	—	—	—	✓
Non-Repudiation	—	—	—	✓

IX. 結論與未來工作

本文提出了 DROS 架構，其核心理念可濃縮為：

“Only cryptographically attributable agents are eligible for interaction.”

透過首創的 By-Execution Certificate (BEC) 機制，DROS 成功解決了當前智能體生態系中粗粒度信任的危機。每一次 Agent 執行皆獲得唯一的、密碼學綁定的短效憑證，從根本上杜絕了憑證竊取、越權存取與重放攻擊的可能性。

展望未來，我們計畫將 DROS 的信任模型擴展為完整的 Agent-to-Agent Trust Fabric——一個去中心化的智能體信任網絡，使跨組織、跨平台的 Agent 協作皆能在密碼學保障下安全進行。這將為下一代「AgentWeb」時代的來臨奠定堅實的基礎設施標準。

References [1] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero Trust Architecture,” NIST Special Publication 800-207, 2020.

[2] R. L. Rivest, A. Shamir, and L. Adleman, “A Method for Obtaining Digital Signatures and Public-Key Cryptosystems,”

TABLE III
智能體協定比較 (AGENT PROTOCOL COMPARISON)

能力	MCP	A2A	DROS
Tool Discovery	✓	✓	—
Agent Communication	✓	✓	—
Identity Verification	—	—	✓
Per-Execution Certificate	—	—	✓
Cryptographic Attribution	—	—	✓
Non-Repudiation	—	—	✓

Communications of the ACM , vol. 21, no. 2, pp. 120–126, 1978.

[3] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, 2015.

[4] European Commission, “Regulation (EU) 2024/1689 —Artificial Intelligence Act,” Official Journal of the European Union , 2024.

[5] M. Wooldridge, An Introduction to MultiAgent Systems , 2nd ed.

Chichester, UK: John Wiley & Sons, 2009.

[6] E. Feldman et al. , “SPIFFE: Secure Production Identity Framework for Everyone,” Cloud Native Computing Foundation (CNCF), 2024.

[7] OWASP Foundation, “OWASP Top 10 for Large Language Model Applications,” 2025. [Online]. Available: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>

[8] Chen, J. (2026). DROS-RFC-004: Ontology & Governance Specification. Dharma Reasoning Operating System Specification, Jun. 2026.

AI 協作與學術誠信宣告

在撰寫本論文與開發原型系統的過程中，作者利用了大型語言模型（包括但不限於 Google Gemini 與 Anthropic Claude 系列）協助進行學術文獻翻譯、程式碼框架生成、以及英文化排版校對。作者對本文的所有邏輯架構、核心演算法設計（包含 Vajra 體系與 PKI 零信任架構）與最終內容承擔完全責任，並已對 AI 生成之內容進行了嚴格的審查與驗證。